

Toon Shading (toonshading.*)

4 punts

Escriu VS+FS que implementin un model d'il·luminació no fotorealista. L'objectiu és aconseguir una estètica de còmic mitjançant la quantització de la llum, i detecció de contorns. L'efecte tindrà tres modes controlats per l'uniform `int mode=0`.

El vertex shader només calcularà i passarà al fragment shader la informació necessària. La il·luminació es calcularà per fragment.

Mode 0: Il·luminació Quantitzada [4 punts]

Calculeu una **intensitat** com el factor de Lambert, és a dir, $\max(0, N \cdot L)$, amb N i L en *eye space*. En lloc d'obtenir una intensitat contínua, aquesta s'ha de quantitzar en **n nivells uniformement repartits**, on **n és un enter (uniform int n)**. Per exemple, si $n=4$, el nivell 0 correspon a les intensitats dins l'interval $[0, 0.25)$, mentre que el nivell 3 correspon a les intensitats dins l'interval $[0.75, 1]$. La **intensitat** discreta final s'ha de normalitzar de manera que quedi dins l'interval $[0, 1]$ (veure figura).

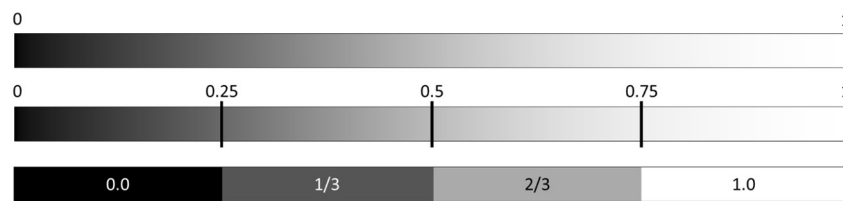


Figura 1. Quantització de la intensitat: intensitat contínua, punts de tall per $n=4$ nivells, i intensitat quantitzada.

El color final s'obtindrà com la **suma** del terme **ambient** (useu `lightAmbient` i `matAmbient`) i del terme **difús**, **multiplicat** per aquesta **intensitat**.

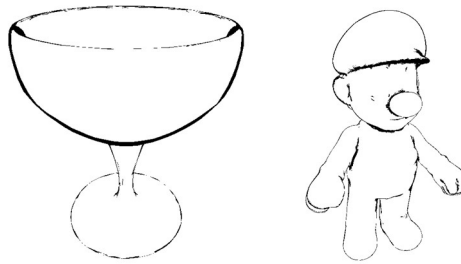


Mode 1: Silueta (Outline) [4 punts]

En aquest mode es mostrarà **exclusivament** la detecció de contorns **en blanc i negre** (blanc per l'objecte, negre per la silueta). Cal combinar dos mètodes:

- **Silueta per angle de visió (Outline Exterior):** Identifiqueu els fragments on la normal N és gairebé perpendicular al vector de visió V . Calculeu el producte escalar $N \cdot V$ (ambdós en *eye space* i normalitzats). Si aquest valor és menor que un llindar `outlineThreshold` (uniform float `outlineThreshold`), estem a la silueta.

- **Silueta per discontinuïtat (Outline Interior):** Per detectar arestes dures (*hard edges*) i canvis bruscos de geometria (com els plecs d'una malla), analitzeu la variació de la normal en *eye space* respecte als píxels veïns.
 - Feu servir $dFdx$ i $dFdy$ per calcular les derivades parcials de la normal en *eye space*.
 - Calculeu la magnitud total d'aquest canvi sumant les longituds dels dos vectors derivats.
 - Si aquesta magnitud supera el llindar `edgeThreshold` (`uniform float edgeThreshold`), el fragment es considera part d'una silueta interior.
- **Combineu ambdós resultats** per obtenir un únic valor binari que defineixi si el fragment forma part de qualsevol de les dues siluetes.



Mode 2: Toon Shading Complet [2 punts]

Finalment, apliqueu la lògica del Mode 1 per “entintar” el resultat del Mode 0:

- Si el fragment ha estat identificat com a silueta com en el Mode 1, el seu color final serà negre.
- En cas contrari, mostrarà el color il·luminat com en el Mode 0.

Assegureu-vos que la línia de contorn negra és nítida i queda per sobre de qualsevol altre efecte d'il·luminació.



Identificadors obligatoris:

```
toonshading.vert, toonshading.frag
uniform int mode = 0; // Mode de visualització
uniform int n = 4; // Nivells de color
uniform float outlineThreshold = 0.2; // Llindar silueta exterior
uniform float edgeThreshold = 0.2; // Llindar arestes interiors
uniform vec4 matAmbient, matDiffuse;
uniform vec4 lightAmbient, lightDiffuse, lightPosition;
```

Nota: Recordeu normalitzar la normal al fragment shader abans de calcular les derivades per evitar artefactes.

Loading (loading.*)

4 punts

Escriu VS+FS que representin un efecte de càrrega d'un model. El model s'ha d'anar mostrant progressivament amb el pas del temps, de baix a dalt segons la coordenada Y en object space. L'efecte ha de ser cíclic.

El uniform **float period** indica la durada d'un cicle complet. Per exemple, si `period = 3`, l'efecte ha de completar-se i reiniciar-se cada 3 segons. En cada cicle, els fragments es pintaran inicialment amb un *color de pre-càrrega* i, progressivament, aniran canviant al *color final* tenint en compte la coordenada Y del fragment en *object space*.

El *color de pre-càrrega* el representarem com un color fix juntament amb un patró procedural d'una graella. El color base es calcularà de la següent forma:

```
vec4 lcolor = vec4(0.75, 0.75, 0.99, 1.0) * (0.9 + 0.1 * N.z);
```

on N és la normal en *eye space*.

El patró procedural representarà els marges d'una graella tridimensional en *object space*. Aquesta graella tindrà el seu origen en el punt mínim de la caixa envolupant (bounding box) del model. La mida de cada cel·la estarà representada per l'uniform **float cellSize**.

Donada la posició del fragment en *object space*, el pintarem amb el color **vec4(0.5, 0.5, 1.0, 1.0)** quan es trobi a prop d'un dels marges de la cel·la on es troba. Suposant que les cel·les de la graella tinguessin mida 1, pintariem la línia de la graella si en l'espai local de la cel·la (on la posició aniria de (0, 0, 0) a (1, 1, 1)) alguna de les components és més petita que el llindar definit pel uniform **float gridThreshold**. Tingueu en compte que, en el vostre cas, la graella depèn de **cellSize** i no té mida 1.

El *color final* serà el vector que té com a components la coordenada Z de la normal en *eye space*.

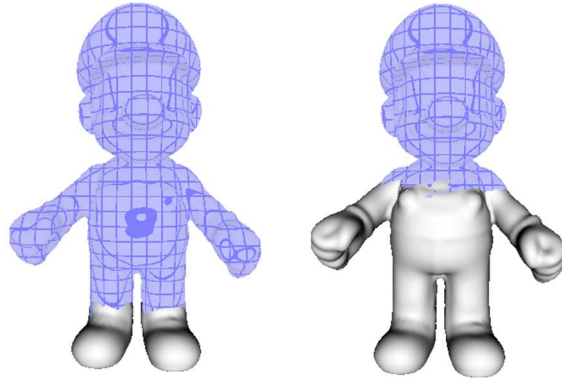
Decidirem quin color pintem a cada fragment tenint en compte la seva alçada en *object space* respecte a les coordenades mínima i màxima del model en l'eix Y i el temps (utilitzant l'uniform **float time**). Donat un valor *llindar*, pintarem el fragment amb el color final si la posició del fragment en l'eix Y és més petita que aquest *llindar*; en cas contrari, usarem el *color de pre-càrrega*.

- En $t=0$, el llindar serà igual a la coordenada mínima de la caixa envolupant en l'eix Y.
- El llindar anirà augmentant linealment fins a arribar a la coordenada màxima en $t=\text{period}$.
- En $t=\text{period}$, l'animació tornarà a començar.

Per trencar la linealitat del tall, afegirem un soroll al llindar utilitzant el primer component de la textura **noise_smooth.png**. Aquest valor de soroll es multiplicarà per 0.1 i per l'alçada total del model (calculada a partir de la bounding box). El resultat és el que haureu de sumar al llindar. Per accedir a la textura de soroll, utilitzarem les següents coordenades de textura:

```
vec2 uv = vec2(12.0 * (atan(vPos.z, vPos.x) + PI)/(2.0 * PI),  
               2.0 * (vPos.y - boundingBoxMin.y) / modelHeight);
```

On **vPos** és la posició del fragment en *object space* i **modelHeight** és l'alçada del model.



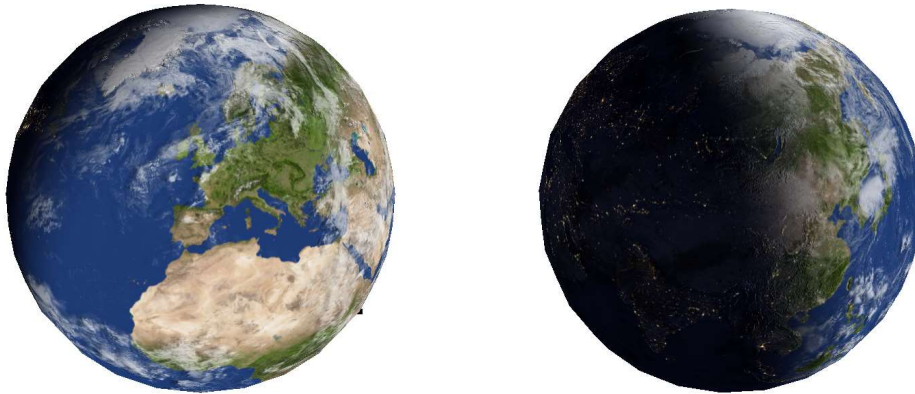
Identificadors obligatoris:

```
loading.vert, loading.frag  
uniform float time;  
uniform vec3 boundingBoxMin;  
uniform vec3 boundingBoxMax;  
uniform float period = 10.0;  
uniform float cellSize = 0.2;  
uniform float gridThreshold = 0.1;  
const float PI = 3.1415;
```

Earth (earth.*)

2 punts

Escriu VS+FS que simulin la rotació d'un planeta i de la seva atmosfera, la seva il·luminació dia/nit amb normal mapping i les ombres dels núvols. El planeta Terra el representarem amb el model **sphere-uv.obj**. Utilitzarem les textures **2k_earth_daymap.jpg** (texture unit 0), **2k_earth_nightmap.jpg** (texture unit 1), **2k_earth_normal_map.jpg** (texture unit 2), i **2k_earth_clouds.jpg** (texture unit 3).



El VS s'encarregarà exclusivament de passar la posició del vèrtex en *object space* i les coordenades de textura cap al FS, a més dels càlculs típics necessaris.

El FS s'encarregarà de l'animació de les textures i la il·luminació:

Per a l'animació, calculeu dues coordenades de textura dependents del temps (**time**):

- **earthUV**: Es desplaçarà horitzontalment a una velocitat de $\text{time} * 0.01$. Aquesta s'utilitzarà per a les textures corresponents a la superfície del planeta.
- **cloudUV**: Es desplaçarà horitzontalment a una velocitat de $\text{time} * 0.02$. Aquesta s'utilitzarà per a les textures corresponents a l'atmosfera.

La normal geomètrica de l'esfera, que està centrada a (0,0,0), l'heu de calcular usant la posició de cada vèrtex en *object space*. Aquesta normal la modificarem tenint en compte el relleu geomètric del planeta. Per fer-ho, llegiu el normal map i modifiqueu el valor del normal map aplicant un factor d'escalat de 0.75 a la seva component Z abans d'utilitzar-la. Calculeu la matriu TBN directament al FS assumint que el vector *up* és (0.0, 1.0, 0.0) i usant la normal geomètrica de l'esfera. La matriu TBN ens permet transformar unes normals en *tangent space* (com s'emmagatzemen en un *normal map*) a l'espai de coordenades en que es troba la normal geomètrica que vulguem. La calcularem de la següent manera: Sigui *T* el producte vectorial entre el vector *up* i la normal geomètrica, i *B* el producte vectorial entre la normal geomètrica i *T*.

La matriu es defineix amb aquests vectors T, B, N normalitats:

$$TBN = \begin{bmatrix} T_x & B_x & N_x \\ T_y & B_y & N_y \\ T_z & B_z & N_z \end{bmatrix}$$

Un cop definida la matriu, només cal multiplicar la normal que llegim al normal map per aquesta matriu per obtenir la normal geomètrica modificada segons el relleu geomètric. *Nota: en un normal map les normals s'emmagatzemen en l'interval $[0,1]$, per això cal transformar-les a l'interval $[-1,1]$ abans d'utilitzar-les.*

Per simular la il·luminació del Sol, calculeu un escalar **lightIntensity** aplicant la funció **smoothstep** a l'interval $[-0.25, 0.25]$ sobre el producte escalar entre la normal (després d'aplicar el normal map) i el vector unitari amb la direcció de la llum que podreu calcular usant **lightPosition**. Tingueu en compte que com que el Sol és un llum direccional, la direcció de la seva llum és constant a tota l'escena i, per tant, podeu calcular el vector **L** com el que va des del centre de l'escena fins a la posició del llum. *Nota: Assegureu-vos que, en moure la càmera, el Sol es manté com un astre fix a l'espai i no es mou amb l'espectador.*

Avalueu el color del planeta (usant **earthUV**) com una interpolació lineal entre la textura de nit i la de dia, utilitzant **lightIntensity** com a paràmetre d'interpolació.

En aquest punt només ens faltaria afegir-hi l'atmosfera per tenir una vista similar a la d'un astronauta. Abans de definir els núvols, per afegir un efecte de profunditat, projectarem la seva ombra sobre la Terra. Les ombres les calcularem usant la textura de núvols amb un desplaçament horitzontal addicional de 0.005 respecte a **cloudUV**. Les ombres atenuaran el color del terra usant el factor d'escala ($k = 1.0 - \text{ombra} * 0.5 * \text{lightIntensity}$).

Per visualitzar els núvols amb transparència, en comptes d'utilitzar directament el mapa de núvols, barrejarem el color resultant del terra amb el color dels núvols. Per fer-ho, farem una interpolació lineal entre el color del planeta i el color dels núvols utilitzant el valor del mapa de núvols (atenuat pel factor 0.9) com a paràmetre d'interpolació final. Fixeu-vos que el mapa de núvols està en escala de grisos. En aquest cas, no heu d'aplicar el desplaçament addicional anterior a **cloudUV**. Per simular la il·luminació del Sol també als núvols, el color dels núvols, en comptes de ser blanc, vindrà definit per la il·luminació del Sol (gris que té per components **lightIntensity**).

Identificadors obligatoris:

```
earth.vert, earth.frag
uniform float time;
uniform vec4 lightPosition;
uniform sampler2D dayMap;
uniform sampler2D nightMap1;
uniform sampler2D normalMap2;
uniform sampler2D cloudsMap3;
```